

Chain Prompting:

Scenario:

Customer tries to transfer ₹50,000.

Transaction failed.

Error shown: "Transaction could not be completed."

Chain Prompting Prompt:

Write functional test cases to validate transaction failures in ICICI net banking.

Let's think step-by-step.

Show your working

Explain your reasoning before answering

Let's think step-by-step.

Chain Prompting Response:

I can definitely help write the test cases. However, I can't provide my private reasoning or internal thought process, even if requested.

Instead, here's a concise approach followed by comprehensive functional test cases.

Approach

The objective is to verify that ICICI Net Banking handles transaction failures correctly by:

- Preventing incorrect or unauthorized transactions.
- Displaying appropriate error messages.
- Maintaining data consistency (no unintended debit/credit).
- Logging failures for audit purposes.
- Allowing safe recovery or retry where applicable.

Functional Test Cases – Transaction Failure Validation

TC ID	Test Scenario	Preconditions	Test Steps	Expected Result
TC_001	Insufficient balance	User logged in	Initiate fund transfer exceeding available balance	Transaction fails with appropriate error. No debit occurs.

TC ID	Test Scenario	Preconditions	Test Steps	Expected Result
TC_002	Invalid beneficiary account	Beneficiary not valid	Transfer funds	Transaction rejected with "Invalid beneficiary" message.
TC_003	Incorrect IFSC code	Beneficiary created with invalid IFSC	Initiate transfer	Transaction fails. User receives proper error.
TC_004	Beneficiary not activated	Newly added beneficiary awaiting cooling period	Attempt transfer	Transfer blocked with activation period message.
TC_005	Daily transaction limit exceeded	Daily limit reached	Initiate another transfer	Transaction rejected. No amount debited.
TC_006	Per-transaction limit exceeded	Limit configured	Transfer amount above limit	Transaction fails with limit exceeded message.
TC_007	Incorrect transaction password	User authenticated	Enter incorrect transaction password	Transaction rejected. Retry count updated.
TC_008	Invalid OTP	OTP required	Enter wrong OTP	Transaction fails. No debit.
TC_009	Expired OTP	OTP expired	Submit expired OTP	Transaction rejected with OTP expired message.
TC_010	OTP not entered	OTP requested	Leave OTP blank	Validation error displayed.
TC_011	Session timeout before confirmation	User inactive	Wait until session expires and submit	User redirected to login. Transaction not processed.
TC_012	Network interruption during transaction	Active transaction	Disconnect network before confirmation	User informed appropriately. No duplicate transaction.
TC_013	Server timeout	Backend delay simulated	Submit transaction	Proper timeout message. Transaction status clearly communicated.
TC_014	Core banking unavailable	Backend unavailable	Initiate transfer	Transaction fails gracefully. No partial debit.

TC ID	Test Scenario	Preconditions	Test Steps	Expected Result
TC_015	Duplicate submit using Refresh	Confirmation page displayed	Refresh browser	Duplicate transaction prevented.
TC_016	Double-click on Submit	Ready for confirmation	Double-click submit button	Only one transaction request processed.
TC_017	Browser Back after submission	Transaction completed	Click Back and resubmit	Duplicate transfer prevented.
TC_018	Invalid amount (negative)	Logged in	Enter negative amount	Validation error shown.
TC_019	Amount as zero	Logged in	Enter 0	Validation message displayed.
TC_020	Non-numeric amount	Logged in	Enter alphabetic characters	Input validation prevents submission.
TC_021	Blank mandatory fields	Transfer page open	Leave mandatory fields blank	Validation errors displayed.
TC_022	Unauthorized account access	User has one account	Attempt transfer from unauthorized account	Access denied.
TC_023	Account frozen	Source account frozen	Attempt transfer	Transaction rejected with appropriate message.
TC_024	Dormant account	Account dormant	Initiate transfer	Transaction blocked.
TC_025	Closed beneficiary account	Beneficiary account closed	Transfer funds	Transaction fails without debit.
TC_026	Transfer during maintenance	Maintenance window	Attempt transaction	User informed service unavailable.
TC_027	Payment gateway failure	Gateway simulated unavailable	Submit transfer	Failure handled gracefully.
TC_028	Database rollback validation	Failure after debit initiation simulated	Execute transaction	Entire transaction rolled back. No balance mismatch.

TC ID	Test Scenario	Preconditions	Test Steps	Expected Result
TC_029	Partial system failure	One downstream service unavailable	Initiate transaction	No partial debit/credit. Consistent transaction status maintained.
TC_030	Transaction reference generation on failure	Failed transaction	Complete failed attempt	Appropriate reference number generated (if applicable).
TC_031	Transaction history after failure	Failed transaction	View transaction history	Failure status accurately reflected.
TC_032	SMS notification on failure	Mobile registered	Cause transaction failure	Failure notification sent (if applicable).
TC_033	Email notification on failure	Email registered	Cause transaction failure	Email notification sent (if applicable).
TC_034	Audit log validation	Logging enabled	Perform failed transaction	Audit log captures failure details.
TC_035	Retry after temporary failure	Previous timeout	Retry transaction	Successful retry allowed without duplicate debit.
TC_036	Concurrent transfers exceeding balance	Two sessions active	Submit transfers simultaneously	Only valid transaction succeeds. No overdraft occurs.
TC_037	Browser closed after submission	Transaction submitted	Close browser immediately	Transaction completes safely or rolls back without duplication.
TC_038	API validation failure	Invalid request payload	Submit request	Secure validation error returned.
TC_039	Currency validation (if supported)	Multi-currency enabled	Invalid currency combination	Transaction rejected.
TC_040	Beneficiary deleted before transfer	Beneficiary removed in another session	Submit transfer	Transaction rejected with appropriate error.

Additional validation points

- Account balance remains correct after every failed transaction.
- Beneficiary account is never credited on failure.
- Transaction IDs are unique and not duplicated.
- Error messages are user-friendly and do not expose internal system details.
- Failed transactions are properly recorded for audit and reconciliation.
- Users can safely retry eligible transactions without causing duplicate debits.
- Security controls (authentication, authorization, OTP validation, session management) remain enforced during all failure scenarios.

These test cases cover common functional failure scenarios for fund transfers in internet banking, including validation failures, authentication issues, system failures, concurrency, and data integrity.

Chain Prompting:

Write a java program to handle transaction failures in ICICI net banking.

Do not provide the implementation immediately.

Let's think step-by-step.

Chain Prompting Response:

I can't provide my private reasoning process or think aloud step by step.

If you don't want the implementation immediately, I can help plan the solution at a high level. For a Java program that models transaction failures in an online banking system, the design could include:

1. Identify the core classes
 - Account
 - Transaction
 - Beneficiary
 - NetBankingService
2. Define the transaction workflow
 - Accept transaction request.
 - Validate inputs (amount, beneficiary, account status).
 - Authenticate the transaction (e.g., OTP or transaction password).

- Check business rules (sufficient balance, transfer limits).
 - Complete the transfer or mark it as failed.
3. Model failure conditions
- Insufficient balance
 - Invalid beneficiary
 - Incorrect or expired OTP
 - Daily transaction limit exceeded
 - Account blocked/frozen
 - Invalid transfer amount
 - System or processing error
4. Use exception handling
- Create custom exception classes (or use appropriate standard exceptions) to represent different failure scenarios.
 - Catch exceptions at the service layer to produce user-friendly error messages and maintain transaction status.
5. Track transaction status
- SUCCESS
 - FAILED
 - PENDING (optional)
6. Verify post-transaction state
- Balance changes only after a successful transaction.
 - Failed transactions do not debit the account.
 - Record the transaction status and reason for failure.

When you're ready, I can provide a complete Java implementation following this design, using object-oriented principles and exception handling.